Documento de Diseño de Software del Motor de Asignación Automática de Horarios de Clases

Documento Maestro del Entregable 2.3 del Módulo 2

Jesús David Beltrán Zárate

Juan Sebastián Albarracín Isaza

Daniel Felipe Delgadillo Sepúlveda

Martín Daniel Porto Gutiérrez

Programa de Ingeniería de Sistemas, Facultad de Ingeniería, Universidad El Bosque

Proyecto Núcleo 2 (15319), 2026-2, Grupo 5

Ing. Samuel Galindo Pinzón

22 de octubre de 2026

Repositorio https://github.com/jesusBeltran79/ProgHorarioClase

Normas APA 7.ª edición

Introducción

Este documento presenta el diseño del motor de asignación automática de horarios que el Grupo 5 construirá en el Módulo 3. Parte del Informe de Ingeniería Parte I entregado en el Módulo 1 y de los dos productos de la actividad De la incertidumbre al diseño. En la Fase 1 el equipo dejó un mapa con cinco incertidumbres y dos prototipos planteados. En la Fase 2 ejecutó esos prototipos y entregó un dossier de pre diseño. Aquí se cierran esas decisiones y cada una se presenta con la evidencia que la respalda.

El documento se lee junto con el Documento de Anexos. Los diagramas de clases, actividades, secuencia y componentes y el diagrama entidad relación están allí con su propia numeración (Figuras A1 a E1) y cada uno se cita desde la sección que lo explica. Las mediciones que aparecen en este texto se pueden repetir con el código que queda en la carpeta motor-horarios del repositorio del equipo. El Anexo F explica cómo hacerlo.

Contextualización Actualizada Respecto al Módulo 1

El problema no cambió. El sistema debe asignar a cada sesión semanal de cada grupo una tupla formada por un docente, un aula y una franja horaria sin violar las restricciones duras del caso (Lopez-Cruz, s.f.-a). El Módulo 1 tomó la generación de horarios como un problema mal estructurado en el sentido de Simon (1973) y lo acotó a un subconjunto que se puede resolver con un método exacto en doce semanas. Ese alcance se mantiene en este documento.

Lo que se Confirmó

La grilla alineada desde las 07:00 con bloques de dos horas (S-01) y la exclusión de los bloques que cruzan el intervalo de 12:00 a 13:00 o superan el cierre (S-02) producen exactamente 32 franjas válidas. Ya no es solo un cálculo en papel. El método generarFranjasValidas de la clase ConfiguracionParametros construye ese dominio a partir de los parámetros y devuelve las mismas 32 franjas que derivó el Módulo 1.

También se confirmó que el backtracking con poda es viable para el tamaño del MVP. Con el conjunto de sustentación de 10 docentes, 5 aulas y 20 asignaturas el motor resuelve las 40

sesiones en pocos milisegundos y con el triple de datos se mantiene por debajo de un décimo de segundo (Sección 5). El espacio de búsqueda sin poda es de 1.600 tuplas por sesión elevado a 40 sesiones, del orden de 1,4 × 10^128 combinaciones. El Módulo 1 reportó 10^127 y la cifra queda corregida aquí. Se confirmó además la separación del Módulo 1 entre restricciones duras (RD), restricciones blandas (RB) y reglas de validación sobre los datos de entrada (RV). Esa separación terminó siendo la base del diseño de clases.

Lo que Cambió y por qué

Los cambios vienen de los prototipos de la Fase 2 y de la revisión que se hizo para este entregable. La Tabla 1 los resume. Ninguno modifica el alcance validado con el docente. Todos precisan cómo se va a construir ese alcance.

Tabla 1

Ajustes del diseño respecto al Módulo 1 y a la Fase 1

Elemento	Planteamiento anterior	Decisión en este documento	Origen del cambio
Contrato de las reglas	Una sola interfaz Restriccion para duras, blandas y el tope de 40 (hipótesis B3 de la Fase 1)	Dos contratos. RestriccionDura para el backtracking y ReglaValidacionEntrada para RV-01 y RV-02 antes de asignar	La fuerza 3 de la Fase 1 se confirmó. RV-01 depende solo del grupo y evaluarla en cada nodo repite un resultado ya conocido
Explicación de un conflicto	Una frase que resume la causa (boceto de la Fase 1)	Lista de causas por código RD-xx con el número de tuplas que bloquea cada una	Prueba del Prototipo 1 en la Fase 2. Un participante leyó solo la primera causa
Técnica de interacción	Arrastrar y soltar sugerido por el documento base	Selección asistida con alternativas filtradas por el motor	Sección 8 de este documento
Persistencia	Solo archivos CSV o JSON	CSV o JSON siguen siendo el formato de entrada y salida. El modelo normalizado del DER guarda inventarios, corridas y conflictos	El entregable exige un DER normalizado y RNF-06 exige saber quién generó cada horario
Verificación de choques	Recorrido de la lista de asignaciones	Mapas de ocupación por franja en EstadoHorario	Medición de la Sección 5.1
Habilitación docente	Supuesto S-06 sin componente propio	Se verifica dentro de RD-04 junto con la disponibilidad	Ambas condiciones definen si un docente puede tomar una sesión

Nota. RD = restricción dura. RV = regla de validación. Las demás decisiones del Módulo 1 se conservan sin cambios.

Hay un ajuste que merece explicación aparte. Las cifras de tiempo del dossier de la Fase 2 no se pudieron reproducir con el código consolidado del motor. Para este documento se repitieron todas las mediciones con semillas fijas y dos perfiles de datos. Los resultados cambian la lectura del riesgo del orden de exploración y se presentan completos en la Sección 5.1.

Arquitectura y Tecnología

El sistema se organiza como un monolito modular en cuatro capas. La capa de presentación es una interfaz web para el coordinador. La capa de aplicación expone una API REST y coordina el flujo. La capa de dominio contiene el validador, el motor y las restricciones. La capa de persistencia lee los archivos de entrada, exporta el horario y guarda el historial de corridas. La Figura E1 del Documento de Anexos muestra los componentes y el nodo donde corre cada uno.

Hu (2023) recomienda escoger la arquitectura por los atributos de calidad que el sistema debe cumplir y no por costumbre. En este caso mandan dos atributos. El primero es el desempeño del motor (RNF-01 y RNF-02) y el segundo es la facilidad para cambiar reglas (RNF-03). Las capas sirven a los dos porque dejan el motor aislado de la interfaz y de los archivos. El costo de las capas es la indirección y por eso el motor trabaja solo en memoria. Los inventarios se leen una vez antes de empezar y el resultado se guarda una vez al terminar. Dentro del ciclo del backtracking no hay ningún acceso a disco ni a red.

Se descartaron dos alternativas. Una arquitectura de microservicios no aporta nada con un solo actor y un solo proceso de cálculo y metería llamadas de red dentro del ciclo más repetido del sistema. Un programa sin capas que lea los archivos desde el motor sería más corto al comienzo, pero cualquier cambio de formato obligaría a tocar el algoritmo.

La tecnología propuesta es Java 21 para el dominio, Spring Boot para la API REST y la autenticación del perfil administrador (RNF-06), Angular con TypeScript para la interfaz web y PostgreSQL para el modelo del DER. Java se mantiene porque el motor ya está escrito y medido en ese lenguaje y porque el equipo lo maneja desde semestres anteriores. El documento base pide

un programa web (Lopez-Cruz, s.f.-a) y por eso la interfaz corre en el navegador. Para el desarrollo local la base puede ser H2 sin cambiar el modelo.

Diseño del Algoritmo de Asignación

El motor trata cada sesión como una variable. El dominio de cada variable son las tuplas posibles de docente, aula y franja. Las restricciones duras deciden qué tuplas son válidas. Russell y Norvig (2021) describen este planteamiento como un problema de satisfacción de restricciones y recomiendan resolverlo con backtracking y una heurística de variable más restringida. El motor sigue esa receta en cuatro pasos y la Figura B1 del Documento de Anexos los muestra como diagrama de actividades.

Primero cuenta para cada sesión cuántas tuplas pasan las restricciones locales RD-04, RD-05 y RD-06. Son locales porque solo miran la tupla y no el horario ya construido. Luego ordena las sesiones de menor a mayor dominio para atender primero las que tienen menos opciones. Después recorre las sesiones en ese orden y para cada una prueba tuplas hasta encontrar una que cumpla todas las restricciones. Las locales se evalúan antes que los choques de docente y de aula porque son más baratas. Si ninguna tupla sirve el motor deshace la última asignación y vuelve a la sesión anterior.

El cuarto paso responde la pregunta de la guía sobre qué pasa cuando se agotan las opciones de aula o de docente. Si el motor recorre todo el árbol sin éxito o supera el tiempo límite configurable recupera la asignación parcial más profunda que alcanzó. Intenta ubicar sobre ella las sesiones que faltan. Para cada sesión que sigue sin lugar evalúa todas las restricciones sin detenerse en la primera falla y cuenta cuántas tuplas bloquea cada código. Con eso arma un ReporteConflicto que lista las causas y las alternativas que solo fallan por un choque con otra sesión. El horario queda marcado como pendiente y la exportación se bloquea porque una asignación parcial no cumple RD-08.

La demostración incluida en el repositorio reproduce el caso del boceto de la Fase 1. El Dr. Gómez es el único docente habilitado para Cálculo I y Cálculo II y solo declara tres franjas

libres, pero los dos grupos suman cuatro sesiones. El motor ubica 7 de las 8 sesiones y explica la que falta con cuatro códigos. RD-04 bloquea 183 tuplas, RD-06 bloquea 64, RD-01 bloquea 9 y RD-02 bloquea 6. Es justo la situación de varias causas a la vez que el boceto no sabía mostrar. La Figura C1 del Documento de Anexos sigue ese mismo caso como diagrama de secuencia.

Análisis de Atributos de Calidad

Escalabilidad

La pregunta de la guía es qué le pasa al diseño si suben los docentes, las aulas y los cursos. El análisis se hace primero paso a paso y luego se contrasta con mediciones. Sea k el factor de escala sobre el conjunto de sustentación. Con k docentes por cada 10, k aulas por cada 5 y k asignaturas por cada 20 el número de franjas no cambia porque F = 32 depende solo de la ventana de oferta.

El dominio de una sesión es $|D| \times |A| \times F = (10k)(5k)(32) = 1.600k^2$ tuplas. Duplicar los datos multiplica por 4 las opciones de cada sesión y triplicarlos las multiplica por 9. La profundidad del árbol es el número de sesiones $N = 40k$. El conteo inicial de la heurística revisa todas las tuplas de todas las sesiones y por eso hace $N \times 1.600k^2 = 64.000k^3$ evaluaciones. Son 64.000 en 1x, 512.000 en 2x y 1.728.000 en 3x. Los contadores del prototipo dieron exactamente esas cifras. La verificación de choques depende de la estructura de EstadoHorario. Si las asignaciones confirmadas se guardan en una lista, revisar un choque cuesta tantas comparaciones como asignaciones existen. Si se guardan mapas de ocupación por franja el costo es constante.

Para medir se ejecutó el motor en Java 21 sobre una máquina virtual de 2 núcleos a 2,1 GHz. Cada escala se corrió con cinco conjuntos generados con semillas distintas después de calentar la máquina virtual de Java y con un límite de 15 segundos por corrida. Se usaron dos perfiles. El perfil holgado imita el conjunto de sustentación. El perfil exigente vuelve escasas las salas con computadores porque solo una de cada cinco aulas los tiene y más de la mitad de los cursos los pide. Se compararon tres variantes del motor. La Tabla 2 resume los resultados y el Anexo F trae cada corrida.

Tabla 2

Tiempo total por corrida en milisegundos según escala, perfil y variante del motor

Escala	Variante	Holgado prom.	Holgado peor	Exigente prom.	Exigente peor
1x (40 ses.)	Orden ingenuo con lista	1,0	1	3.006,6	15.024 (límite)
1x (40 ses.)	Más restringida con lista	5,4	10	2,8	3
1x (40 ses.)	Más restringida con mapas	3,4	4	2,2	3
2x (80 ses.)	Orden ingenuo con lista	5,8	7	6,4	12
2x (80 ses.)	Más restringida con lista	23,8	31	19,0	29
2x (80 ses.)	Más restringida con mapas	17,2	31	19,0	22
3x (120 ses.)	Orden ingenuo con lista	11,4	12	16,6	19
3x (120 ses.)	Más restringida con lista	33,4	42	53,0	75
3x (120 ses.)	Más restringida con mapas	37,0	45	46,4	53

Nota. Promedio y peor caso de cinco corridas por celda. Todas las corridas terminaron con el horario completo salvo la marcada como límite, que agotó los 15 segundos sin encontrar solución. Datos en resultados/mediciones_1x_2x_3x.csv y resultados/mediciones_exigente.csv del repositorio.

La primera lectura es que el diseño cumple los requisitos con mucho margen. La peor corrida con la heurística en 3x tardó 75 milisegundos. RNF-01 pide 60 segundos sobre el conjunto de sustentación y RNF-02 pide procesar el triple sin cambiar la estructura de datos. Ambos se cumplen.

La segunda lectura cambia lo que el equipo creía después de la Fase 2. En promedio el orden ingenuo es el más rápido porque no paga el conteo inicial. Su problema está en la cola. En el perfil exigente una de las cinco instancias de 1x hizo que el orden ingenuo explorara 488 millones de nodos y agotara el límite de 15 segundos. La misma instancia con la heurística se resolvió en 3 milisegundos y 25.656 nodos. El orden ingenuo ubicó primero cursos comunes en las pocas salas con computadores y luego no tuvo dónde poner los cursos que sí las necesitaban. Un coordinador no puede predecir qué conjunto de datos va a caer en ese caso. Por eso se mantiene la heurística de la sesión más restringida. Se acepta pagar unos milisegundos más en promedio para que el peor caso quede acotado.

La tercera lectura identifica el primer cuello de botella. Con la heurística activa el conteo inicial consume entre el 50 y el 81 por ciento del tiempo total y en 2x y 3x nunca baja del 65 por

ciento. Es la parte que crece más rápido porque sigue a 64.000k³. Los choques pesan menos de lo esperado. Con lista el motor hizo en promedio 746.018 comparaciones de choque en 3x holgado. Con mapas por franja hizo 13.746, unas 54 veces menos. A estas escalas el tiempo casi no cambia porque cada comparación es muy barata. Aun así se adoptan los mapas porque eliminan un costo que crece con el número de asignaciones y solo ocupan dos arreglos de 32 conjuntos.

De ahí sale una decisión para el Módulo 3. El conteo inicial se puede factorizar porque RD-04 solo depende del docente y la franja y RD-05 y RD-06 solo dependen del aula. Basta contar por cada franja cuántos docentes y cuántas aulas sirven y multiplicar. El costo baja de N × |D| × |A| × F a N × F × (|D| + |A|). En 3x serían 172.800 operaciones en lugar de 1.728.000. Queda pendiente porque exige que cada restricción declare de qué elemento de la tupla depende y esa decisión toca el contrato de RestriccionDura.

Si el crecimiento viene por el lado de las franjas el efecto es lineal. Ampliar la ventana de oferta aumenta F y con él el dominio de cada sesión, pero no cambia la profundidad del árbol. Ningún cambio de escala obliga a modificar clases. Solo cambian los datos de entrada y los parámetros.

Modificabilidad

La guía pregunta qué tan fácil es cambiar una regla de negocio como el tope de 40 estudiantes o la franja horaria. Para responder con evidencia se aplicaron tres cambios reales sobre el código y se ejecutó el motor después de cada uno. La salida está en resultados/prueba_de_cambio.txt y la Tabla 3 resume qué se tocó.

El primer cambio baja el tope de 40 a 35 estudiantes con la misma tolerancia. Es un valor de ConfiguracionParametros que el coordinador edita en la pantalla de parámetros (RF-08). El umbral efectivo pasa de 44 a 38 y el validador marca tres grupos con RV-01 que antes pasaban. Ninguna clase cambia. El segundo cambio adelanta el cierre de lunes a viernes de las 22:00 a las 20:00. El método generarFranjasValidas recalcula el dominio y quedan 27 franjas en lugar de 32. La disponibilidad de los docentes llega en los archivos como pares de día y hora y

CargadorInventariosCSV la traduce al índice de franja al cargar. Por eso los archivos no se rehacen cuando cambia la grilla. El tercer cambio agrega una regla dura que no existía. RD-09 prohíbe que dos sesiones del mismo grupo queden el mismo día. Se creó la clase RestriccionMismoDiaRD09 y se registró en la lista de restricciones. El motor generó el horario completo de 40 sesiones sin ningún grupo con dos sesiones el mismo día y RD-09 rechazó 360 tuplas en el camino.

Tabla 3

Elementos que cambian y elementos que no cambian en la prueba de cambio

Elemento del diseño	Tope 40 a 35	Cierre 22:00 a 20:00	Nueva RD-09	Comentario
ConfiguracionParametros	Un valor	Un valor	No	Los valores llegan desde la pantalla de parámetros
RestriccionMismoDiaRD09	No aplica	No aplica	Clase nueva	Implementa RestriccionDura
Lista de restricciones en el ensamblaje	No	No	Una línea	ControladorHorarios inyecta la lista al motor
MotorAsignacion	No	No	No	Cero líneas modificadas según la comparación de versiones
ReglaTopeMatriculadosRV01	No	No	No	Lee el tope desde la configuración
RD-01, RD-02, RD-04, RD-05 y RD-06	No	No	No	Cada regla está aislada en su clase
CargadorInventariosCSV	No	No	No	Traduce día y hora al índice de franja vigente
EstadoHorario	No	No	No	Crea tantos mapas como franjas reciba

Nota. La comparación de versiones entre el código antes y después de RD-09 solo muestra la clase nueva y la línea de registro. Esa es la verificación que exige RNF-03.

Justificación del Patrón de Diseño Strategy

Contexto y Problema

Durante la generación del horario (RF-06) MotorAsignacion evalúa miles de tuplas candidatas y cada una debe pasar las restricciones duras RD-01, RD-02, RD-04, RD-05 y RD-06. Cuando una tupla falla el sistema debe saber cuál regla la rechazó porque RF-07 y RNF-04 exigen reportar el código. Además RNF-03 exige que una regla nueva se agregue creando una

clase sin modificar el motor. El problema es cómo organizar esas reglas para que el algoritmo no dependa de cuántas son ni de cómo se calcula cada una. Ortega-Arjona (s.f.) insiste en que un patrón solo sirve si el problema del proyecto se parece al que el patrón resuelve. Este se parece mucho. Gamma et al. (1994) proponen Strategy para una familia de algoritmos intercambiables que un contexto usa a través de una interfaz común, y aquí cada restricción es un algoritmo de verificación distinto que el motor usa siempre de la misma forma.

Fuerzas que se Oponen

La primera fuerza enfrenta la extensibilidad con el desempeño. Encapsular cada regla en un objeto deja el motor cerrado a cambios, pero agrega una llamada polimórfica dentro del ciclo más repetido. La segunda enfrenta la explicabilidad con la compacidad. Una sola condición booleana dentro del motor ocupa pocas líneas, pero no dice cuál parte falló. La tercera enfrenta la uniformidad con las diferencias reales entre reglas. Todas las duras responden sí o no, pero unas solo miran la tupla y otras miran el horario completo, y las reglas de validación ni siquiera dependen de la tupla.

Solución

La interfaz RestriccionDura define getCodigo, esLocal y esValida. Cada restricción es una clase concreta e inmutable. MotorAsignacion recibe por constructor la lista de estrategias, la ordena para evaluar primero las locales y la recorre sin conocer las clases concretas. Cuando una estrategia responde falso el motor registra su código y descarta la tupla. El método esLocal resuelve la tercera fuerza porque permite que el conteo inicial use solo las reglas que no dependen del horario. Las reglas RV quedaron fuera de esta familia con su propio contrato ReglaValidacionEntrada. Esa separación corrige la hipótesis de la Fase 1, que proponía una sola interfaz para todo. La Figura A1 del Documento de Anexos muestra la estructura.

Qué Pasaría sin el Patrón

Sin Strategy el método de backtracking tendría cinco condiciones encadenadas que leen directamente los atributos de Docente, Aula, Grupo y Curso. Agregar RD-09 obligaría a editar y

volver a probar la clase más delicada del sistema y RNF-03 dejaría de cumplirse. El conteo inicial tendría que repetir esas mismas condiciones en otro método y cualquier corrección habría que hacerla en dos sitios. Para reportar el código de la regla que falló habría que poner un registro manual en cada rama.

Qué Pasaría con Otro Patrón

Chain of Responsibility (Gamma et al., 1994) encadenaría las reglas y cada una decidiría si pasa la petición a la siguiente. En este problema ninguna regla puede aprobar una tupla por sí sola y ninguna debe alterar el flujo, así que la cadena traslada a cada clase una decisión que no le corresponde y hace más difícil saber en qué eslabón se detuvo. Template Method fijaría el orden de las verificaciones en una clase base y cada variación sería una subclase del motor. Eso impide activar o desactivar reglas por configuración y obliga a usar herencia donde basta la composición (Hu, 2023).

Beneficios y Costos Medidos

Los beneficios son concretos. RNF-03 quedó verificado con RD-09. RNF-04 se cumple porque cada estrategia trae su código. La misma lista sirve para el conteo inicial, para el backtracking y para el diagnóstico de conflictos. Cada regla se puede probar sola con datos simulados. El costo es mantener una clase por regla y pagar una llamada polimórfica por evaluación. Ese costo no se aisló con una medición separada en esta versión, pero con la variante elegida el tiempo total de una corrida completa en 3x incluyendo todas esas llamadas fue de 53 milisegundos en el peor caso. Frente al límite de 60 segundos de RNF-01 el costo no pesa en la decisión.

Principios SOLID Aplicados

El diseño aplica dos principios de forma explícita. Ambos se ven en clases reales del código y no solo en el diagrama.

Principio Abierto Cerrado

Martin (2003) lo formula como que un módulo debe estar abierto a la extensión y cerrado a la modificación. MotorAsignacion cumple esa idea porque su lógica de recorrido no cambia cuando aparece una regla nueva. El fragmento muestra el único lugar donde el motor toca las reglas.

```
public final class MotorAsignacion {
    private final List<RestriccionDura> restricciones;

    private boolean cumpleTodas(Asignacion candidata, EstadoHorario estado) {
        for (RestriccionDura r : restricciones) {
            if (!r.esValida(candidata, estado, config)) {
                estado.registrarRechazo(candidata.getSesion(), r.getCodigo());
                return false;
            }
        }
        return true;
    }
}
```

La extensión vive en otra clase. RD-09 se agregó así y el motor no cambió (Sección 5.2).

```
public final class RestriccionMismoDiaRD09 implements RestriccionDura {
    public String getCodigo() { return "RD-09"; }
    public boolean esLocal() { return false; }
    public boolean esValida(Asignacion c, EstadoHorario e, ConfiguracionParametros cfg)
{
        for (Asignacion a : e.getAsignaciones())
            if (a.getSesion().getGrupo() == c.getSesion().getGrupo()
                    && a.getFranja().getDia() == c.getFranja().getDia()) return false;
        return true;
    }
}
```

Principio de Inversión de Dependencias

Las clases que deciden la política del sistema no dependen de las que implementan los detalles. MotorAsignacion depende de la abstracción RestriccionDura y ValidadorEntrada depende de ReglaValidacionEntrada. Ninguna de las dos crea las reglas. Las recibe por constructor y el ensamblaje lo hace ControladorHorarios. Eso permite probar el motor con restricciones simuladas, como propuso Daniel Delgadillo en su ficha de la Fase 1.

```
List<RestriccionDura> reglas = List.of(
        new RestriccionDisponibilidadDocenteRD04(), new RestriccionIdoneidadAulaRD05(),
        new RestriccionCapacidadAulaRD06(), new RestriccionChoqueDocenteRD01(),
        new RestriccionChoqueAulaRD02());
```

```
MotorAsignacion motor = new MotorAsignacion(reglas, configuracion, limiteMs);
ValidadorEntrada validador = new ValidadorEntrada(List.of(
        new ReglaTopeMatriculadosRV01(), new ReglaCierreMinimoRV02()));
```

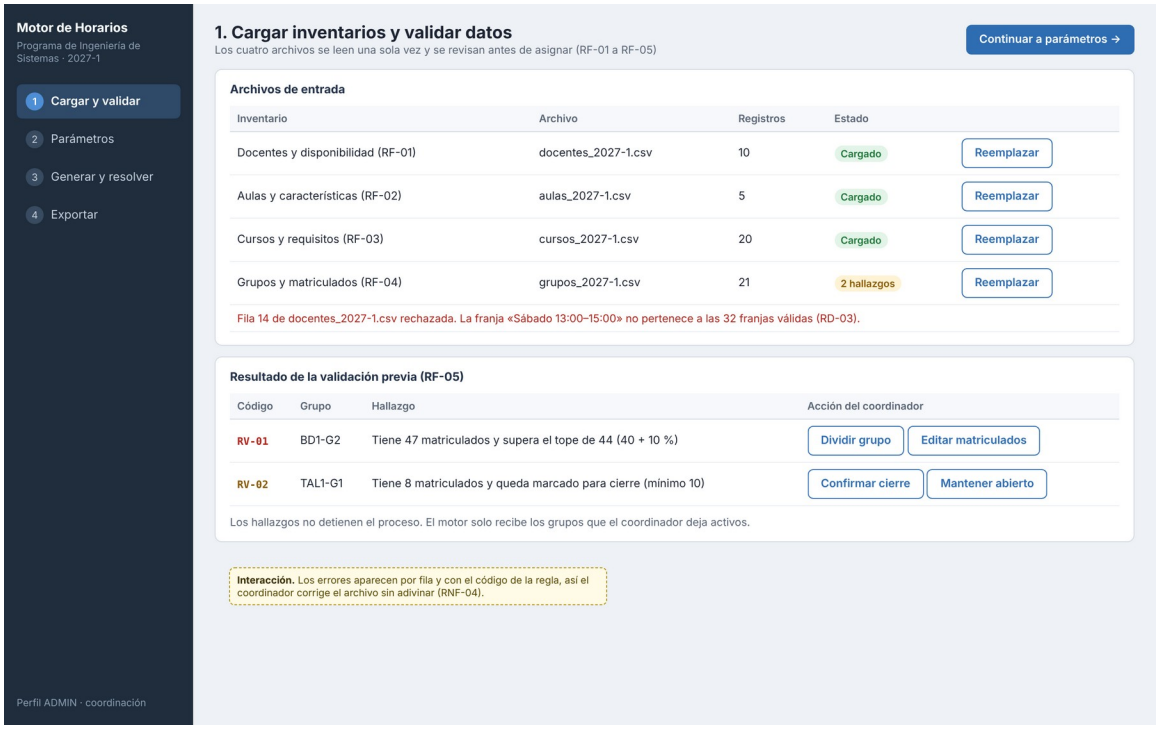
El principio de responsabilidad única aparece como consecuencia. Cada clase de restricción tiene una sola razón para cambiar y la persistencia está separada del motor. Por eso un cambio en el formato de los archivos no toca el algoritmo.

Interfaz de Usuario y Técnica de Interacción

La interfaz tiene tres pantallas que siguen el flujo del coordinador. Primero carga y valida los datos. Luego revisa los parámetros. Al final genera el horario y resuelve lo que quede pendiente. Las Figuras 1 a 3 son wireframes de fidelidad media. Siguen la idea de Lopez-Cruz (s.f.-b) de usar la fidelidad que pide la pregunta y por eso muestran estructura y flujo, no el diseño gráfico final.

Figura 1

Pantalla de carga de inventarios y validación previa

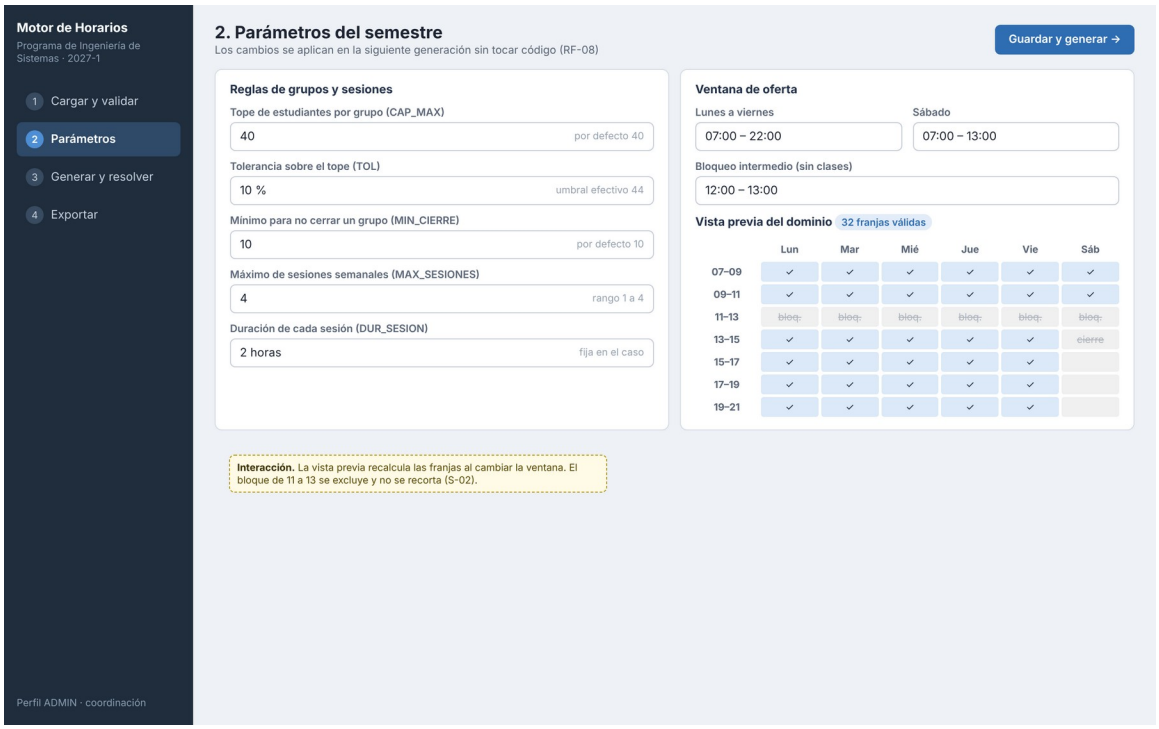


Nota. Muestra RF-01 a RF-05. Los rechazos aparecen por fila y con el código de la regla.

En la pantalla de carga el coordinador sube los cuatro archivos y ve de inmediato los registros rechazados con su motivo. Los hallazgos de RV-01 y RV-02 no detienen el proceso. Se muestran con acciones concretas para que el coordinador decida si divide un grupo, corrige un dato o confirma un cierre antes de generar.

Figura 2

Pantalla de parámetros del semestre

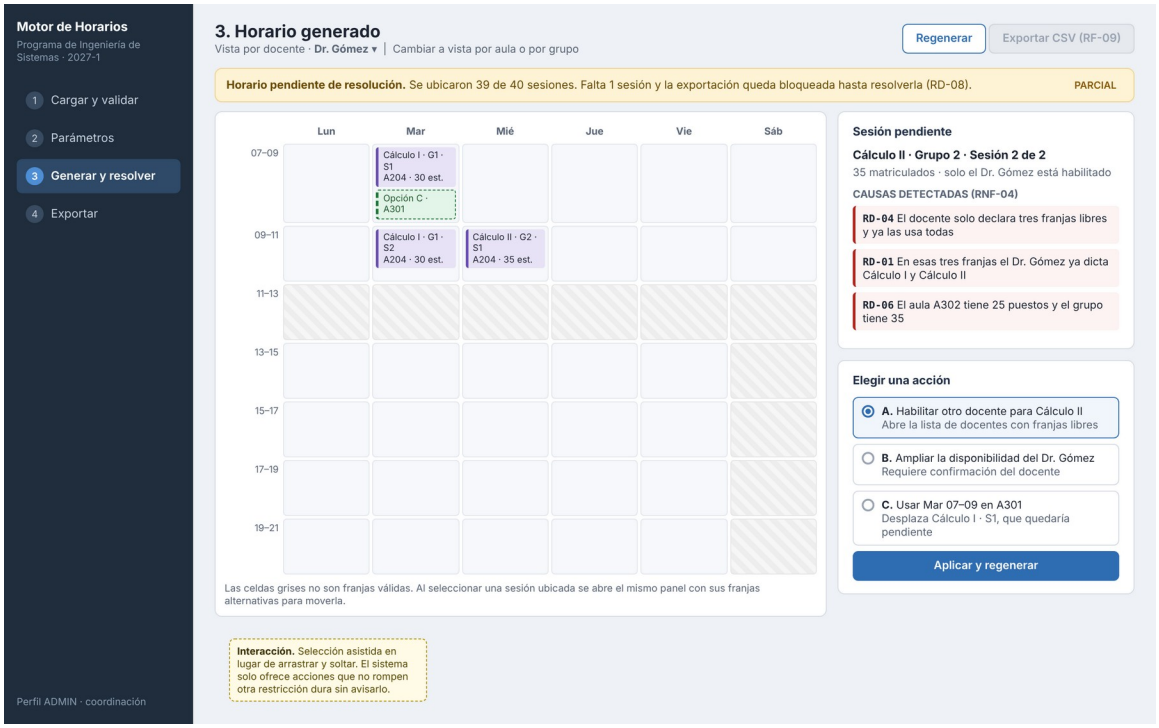


Nota. Muestra RF-08. La vista previa recalcula las franjas válidas cuando cambia la ventana de oferta.

La pantalla de parámetros expone los valores de S-04 y la ventana de oferta. La vista previa deja ver de inmediato cómo queda el dominio de franjas. Así un cambio como el de la Sección 5.2 se entiende antes de generar.

Figura 3

Pantalla del horario generado con una sesión pendiente



Nota. Muestra RF-06, RF-07 y RNF-04 con el caso del Dr. Gómez de la Sección 4.

La pantalla principal muestra la grilla semanal con las 32 franjas válidas y deja en gris las que no existen. Se puede ver por docente, por aula o por grupo. Cuando una sesión queda pendiente aparece un panel con las causas por código y una lista de acciones posibles. Al seleccionar una sesión ya ubicada se abre el mismo panel con sus franjas alternativas para moverla. Así se cumple la edición del horario sin arrastrar bloques.

Por qué Selección Asistida y no Arrastrar y Soltar

El documento base sugiere arrastrar y soltar o la mejor alternativa que proponga el equipo (Lopez-Cruz, s.f.-a). El equipo eligió la selección asistida por tres razones. La primera es que arrastrar deja soltar una sesión en una celda inválida y el sistema solo puede explicar el error después. Con la selección asistida el motor filtra antes y solo ofrece lo que se puede hacer. La segunda viene del Prototipo 1 de la Fase 2. Según el registro del equipo un participante intentó arrastrar y terminó la tarea con el selector, y el promedio de los cuatro participantes quedó por

debajo de tres minutos. La tercera es práctica. La selección funciona igual con mouse, teclado o pantalla táctil y es más sencilla de construir y de probar en una aplicación web. El costo es una interacción menos directa. Se compensa resaltando en la grilla la alternativa elegida antes de confirmarla.

Matriz de Trazabilidad Integral

La Tabla 4 extiende la matriz del Módulo 1. Cada condición queda unida al requisito que la atiende y al componente del diseño que la cumple. La última columna anticipa la prueba del Módulo 3 que cerrará la cadena hasta el código. Ninguna restricción dura queda sin componente y ningún requisito funcional queda sin origen.

Tabla 4

Matriz de trazabilidad entre condiciones del Módulo 1, requisitos y componentes del diseño

ID	Condición del M1	Tipo	Requisitos del M1	Componente del diseño M2	Cómo se verifica en M3
RD-01	Choque de docente	Dura	RF-06, RF-07, RNF-04	RestriccionChoqueDocenteRD01 y mapas de EstadoHorario	Prueba unitaria y prueba de ruptura con dos sesiones del mismo docente
RD-02	Choque de aula	Dura	RF-06, RF-07, RNF-04	RestriccionChoqueAulaRD02 y mapas de EstadoHorario	Prueba unitaria y prueba de ruptura con dos sesiones en la misma aula
RD-03	Solo las 32 franjas válidas	Dura	RF-06, RF-08	ConfiguracionParametros.generarFranjasValidas y FranjaHoraria	Prueba del generador con la ventana por defecto y con cierre a las 20:00
RD-04	Disponibilidad del docente	Dura	RF-01, RF-06, RNF-04	RestriccionDisponibilidadDocenteRD04 y Docente.estaDisponibleEn	Prueba unitaria con disponibilidad parcial
RD-05	Características del aula	Dura	RF-02, RF-03, RF-06, RNF-04	RestriccionIdoneidadAulaRD05 y Aula.cumpleRequisitos	Curso que exige computadores frente a aula sin ellos
RD-06	Capacidad mayor o igual a matriculados	Dura	RF-02, RF-04, RF-06, RNF-04	RestriccionCapacidadAulaRD06	Grupo de 35 frente a aula de 25
RD-07	Entre 1 y MAX_SESIONES sesiones de 2 h	Dura	RF-03, RF-08	CargadorInventariosCSV, Curso y Sesion	Archivo con un curso de 5 sesiones que debe rechazarse

ID	Condición del M1	Tipo	Requisitos del M1	Componente del diseño M2	Cómo se verifica en M3
RD-08	Toda sesión con una tupla	Dura	RF-06, RF-07, RNF-01, RNF-02	MotorAsignacion, ResultadoAsignacion y ReporteConflicto	Caso sobrerrestringido del Dr. Gómez con exportación bloqueada
RV-01	Tope de 44 matriculados	Validación	RF-04, RF-05, RF-08, RNF-04	ReglaTopeMatriculadosRV01 y ValidadorEntrada	Grupo de 47 y cambio de tope a 35
RV-02	Cierre con menos de 10	Validación	RF-04, RF-05, RF-08, RNF-04	ReglaCierreMinimoRV02 y ValidadorEntrada	Grupo de 8 matriculados
RB-01 a RB-04	Preferencias blandas	Blanda	Sin requisito en el MVP	Sin componente. Si se agregan usarían una interfaz RestriccionBlanda separada	No aplica en el MVP
RE-01 a RE-04	Exclusiones declaradas	Fuera de alcance	Sin requisito	Fuera de la frontera del sistema	No aplica
Operativa	Exportar el horario	Operativo	RF-09	ExportadorHorarioCSV	Archivo con curso, grupo, docente, aula, día y franja
Operativa	Acceso restringido	Seguridad	RNF-06	Componente de seguridad de Spring Boot y tabla usuario	Intento de generar sin sesión de administrador
Operativa	Uso por un no técnico	Usabilidad	RNF-05	VistaCoordinador con las tres pantallas	Ciclo completo en 10 minutos o menos con el manual

Nota. Los requisitos RF-01 a RF-09 y RNF-01 a RNF-06 son los del Informe de Ingeniería Parte I. Las condiciones operativas no vienen del caso de estudio y se declaran para que ningún requisito quede huérfano.

Modelo de Datos

El diagrama entidad relación de la Figura D1 del Documento de Anexos tiene 18 tablas y está en tercera forma normal. Cada atributo depende solo de la clave de su tabla. Las relaciones de muchos a muchos se resolvieron con tablas intermedias como disponibilidad_docente, aula_caracteristica, curso_requisito_aula y habilitacion_docente. Así una característica de aula o una franja disponible no se repiten como texto dentro de otra tabla. Todas las columnas tienen tipo definido y el Anexo D trae el diccionario completo.

La integridad referencial se cuida con claves foráneas en todas las relaciones y con restricciones CHECK sobre los valores que el caso acota, como la capacidad positiva o el número de sesiones entre 1 y 4. Hay una decisión que vale la pena resaltar. La tabla asignacion tiene dos restricciones UNIQUE, una sobre ejecución, docente y franja y otra sobre ejecución,

aula y franja. Con eso la base de datos rechaza por su cuenta un horario con choques aunque llegue por un camino distinto del motor. RD-01 y RD-02 quedan protegidas en dos capas.

Los archivos CSV o JSON siguen siendo el formato de intercambio que pidió el Módulo 1. Cada archivo de entrada corresponde a una tabla del modelo y el Anexo D muestra sus columnas. La disponibilidad del docente se escribe como pares de día y hora y no como índices. Esa decisión responde la pregunta que dejó abierta Juan Sebastián Albarracín en su memoria de la Fase 2 y es la que permite cambiar la grilla sin rehacer los archivos.

Decisiones Pendientes Hacia el Módulo 3

Un pendiente es válido si dice qué falta para cerrarlo. La Tabla 5 recoge los tres que quedan abiertos con su responsable.

Tabla 5

Decisiones pendientes y condición de cierre

Pendiente	Qué falta para cerrarlo	Responsable
Conteo inicial factorizado	Agregar al contrato de RestriccionDura el elemento de la tupla del que depende cada regla y medir de nuevo 1x, 2x y 3x	Daniel Felipe Delgadillo Sepúlveda
Orden dinámico de sesiones	Comparar el orden fijo actual con un orden que se recalcula en cada nivel sobre el perfil exigente	Daniel Felipe Delgadillo Sepúlveda
Pruebas con usuarios de la interfaz web	Repetir la prueba del Prototipo 1 sobre la interfaz construida con al menos cuatro personas	Martín Daniel Porto Gutiérrez

Autoevaluación del Equipo

Lo que más facilitó esta entrega fue lo aprendido en Programación Orientada a Objetos y en Estructuras de Datos. Lo primero permitió expresar las reglas como interfaces y clases intercambiables. Lo segundo permitió razonar el costo de recorrer una lista frente a consultar un mapa y anticipar con el análisis paso a paso las cifras que luego dieron los contadores. Bases de Datos fue la base del DER y de la decisión de proteger los choques con restricciones UNIQUE. Ingeniería de Software dio el vocabulario de atributos de calidad para elegir la arquitectura. El tema donde el equipo siente que le falta profundidad es la propagación de restricciones, como la

verificación hacia adelante y la consistencia de arcos, que podría reducir el retroceso en casos más difíciles que los medidos. También falta práctica en pruebas automatizadas con JUnit y en pruebas de carga, que serán necesarias en el Módulo 3.

Declaración de Uso de Inteligencia Artificial

Para este entregable se usó Claude, un asistente de inteligencia artificial de Anthropic, en una conversación de trabajo con un integrante del equipo. Se usó para redactar borradores de las secciones del Documento Maestro a partir del Módulo 1, la Fase 1 y el dossier de la Fase 2, para escribir el código del motor que se usó en las mediciones, para generar las fuentes de los diagramas en PlantUML y los wireframes, y para ejecutar las corridas de medición. La Tabla 6 resume qué se revisó por parte del equipo.

Tabla 6

Uso de inteligencia artificial y validación del equipo

Tarea	Uso de la herramienta	Validación del equipo
Código del motor	Escritura de las clases del dominio, el motor, las restricciones y los programas de medición	Compilar y ejecutar en un equipo propio, revisar cada restricción contra la Tabla 3 del M1 y repetir las corridas
Mediciones	Ejecución de las corridas con semillas fijas y resumen de resultados	Repetir las mediciones con los comandos del Anexo F y confirmar que las cifras coinciden
Diagramas y DER	Fuentes PlantUML de clases, actividades, secuencia, DER y componentes	Revisar notación UML, multiplicidades y coherencia con el código
Redacción	Borradores del texto y de la matriz de trazabilidad	Leer cada sección, ajustar las decisiones que no compartan y verificar referencias

Nota. La herramienta falla con frecuencia en la lógica combinatoria de horarios según los lineamientos del curso. La responsabilidad sobre cada decisión y la capacidad de defenderla en la verificación individual es del equipo.

Referencias

Bardadym, V. A. (1996). Computer-aided school and university timetabling: The new wave. En E. Burke y P. Ross (Eds.), *Practice and theory of automated timetabling* (Lecture Notes in Computer Science, Vol. 1153, pp. 22–45). Springer. https://doi.org/10.1007/3-540-61794-9_50

Bass, L., Clements, P. y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Beltrán Zárate, J. D., Albarracín Isaza, J. S., Delgadillo Sepúlveda, D. F. y Porto Gutiérrez, M. D. (2026a). *Informe de ingeniería parte I. Requisitos, alcance y viabilidad* [Informe de curso no publicado]. Universidad El Bosque.

Beltrán Zárate, J. D., Albarracín Isaza, J. S., Delgadillo Sepúlveda, D. F. y Porto Gutiérrez, M. D. (2026b). *De la incertidumbre al diseño. Dossier de pre diseño, fase 2* [Informe de curso no publicado]. Universidad El Bosque.

Burke, E., Jackson, K., Kingston, J. H. y Weare, R. (1997). Automated university timetabling: The state of the art. *The Computer Journal*, *40*(9), 565–571. https://doi.org/10.1093/comjnl/40.9.565

Carter, M. W. y Laporte, G. (1998). Recent developments in practical course timetabling. En E. Burke y M. Carter (Eds.), *Practice and theory of automated timetabling II* (Lecture Notes in Computer Science, Vol. 1408, pp. 3–19). Springer. https://doi.org/10.1007/BFb0055878

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Hu, C. (2023). *An introduction to software design: Concepts, principles, methodologies, and techniques*. Springer. https://doi.org/10.1007/978-3-031-28311-6

Lopez-Cruz, O. (s.f.-a). *Programación de horarios de clases* [Documento base del reto]. Universidad El Bosque.

Lopez-Cruz, O. (s.f.-b). *Some notes on prototypes* [Material de estudio del Módulo 2]. Universidad El Bosque.

Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.

Ortega-Arjona, J. L. (s.f.). *Software patterns* (cap. 1) [Material de estudio del Módulo 2]. Universidad Nacional Autónoma de México.

Russell, S. y Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.

Simon, H. A. (1973). The structure of ill structured problems. *Artificial Intelligence, 4*(3–4), 181–201. https://doi.org/10.1016/0004-3702(73)90011-8